

CLOUD COMPUTING: SECURITY

Module 7: Security, Identity & Data Protection

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

- 1 Security Foundations
- 2 Encryption & Data Protection
- 3 Network & Application Security
- 4 Audit & Compliance



Outline

- 1 Security Foundations
- 2 Encryption & Data Protection
- 3 Network & Application Security
- 4 Audit & Compliance



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- Defense in Depth: multiple layers of protection
- Least Privilege: grant minimum permissions needed
- Security by Design: build security into systems
- Encryption: protect data at rest and in transit
- Patching: keep systems up to date



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- **Defense in Depth:** multiple layers of protection
- Least Privilege: grant minimum permissions needed
- Security by Design: build security from the start
- Encryption: protect data at rest and in transit
- Monitoring: detect anomalies early



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- **Defense in Depth:** multiple layers of protection
- **Least Privilege:** grant minimum permissions needed
- **Security by Design:** build security from the start
- **Encryption:** protect data at rest and in transit
- **Monitoring:** detect anomalies early



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- **Defense in Depth:** multiple layers of protection
- **Least Privilege:** grant minimum permissions needed
- **Security by Design:** build security from the start
- **Encryption:** protect data at rest and in transit
- **Monitoring:** detect anomalies early



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- **Defense in Depth:** multiple layers of protection
- **Least Privilege:** grant minimum permissions needed
- **Security by Design:** build security from the start
- **Encryption:** protect data at rest and in transit
- **Monitoring:** detect anomalies early



Security is Everyone's Responsibility

Shared Responsibility Model

In cloud, **both** provider and customer have security roles.

Provider: physical, network, hypervisor.

Customer: data, access, application.

Core Security Principles:

- **Defense in Depth:** multiple layers of protection
- **Least Privilege:** grant minimum permissions needed
- **Security by Design:** build security from the start
- **Encryption:** protect data at rest and in transit
- **Monitoring:** detect anomalies early



Authentication vs. Authorization

Authentication

- *Who* are you?
- Prove identity
- Username + password
- Multi-factor (MFA)
- OAuth, SAML tokens

Authorization

- *What* can you do?
- Assign permissions
- Role-based (RBAC)
- Attribute-based (ABAC)
- Policy evaluation

Both are Required

Authentication without authorization is *useless*. Authorization assumes correct authentication.



Authentication vs. Authorization

Authentication

- *Who* are you?
- Prove identity
- Username + password
- Multi-factor (MFA)
- OAuth, SAML tokens

Authorization

- *What* can you do?
- Assign permissions
- Role-based (RBAC)
- Attribute-based (ABAC)
- Policy evaluation

Both are Required

Authentication without authorization is *useless*. Authorization assumes correct authentication.



Authentication vs. Authorization

Authentication

- *Who* are you?
- Prove identity
- Username + password
- Multi-factor (MFA)
- OAuth, SAML tokens

Authorization

- *What* can you do?
- Assign permissions
- Role-based (RBAC)
- Attribute-based (ABAC)
- Policy evaluation

Both are Required

Authentication without authorization is *useless*. Authorization assumes correct authentication.



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- Use Roles: don't assign permissions to individual users (use groups)
- Regular Audits: review who has access to what
- Service Accounts: separate identity for applications
- MFA Everywhere: especially for privileged accounts
- Temporary Credentials: short-lived tokens, not long-lived keys



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- **Use Roles:** don't assign permissions to individual users (use groups)
- Regular Audits: review who has access to what
- Service Accounts: separate identity for applications
- MFA Everywhere: especially for privileged accounts
- Temporary Credentials: short-lived tokens, not long-lived keys



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- **Use Roles:** don't assign permissions to individual users (use groups)
- **Regular Audits:** review who has access to what
- **Service Accounts:** separate identity for applications
- **MFA Everywhere:** especially for privileged accounts
- **Temporary Credentials:** short-lived tokens, not long-lived keys



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- **Use Roles:** don't assign permissions to individual users (use groups)
- **Regular Audits:** review who has access to what
- **Service Accounts:** separate identity for applications
- MFA Everywhere: especially for privileged accounts
- Temporary Credentials: short-lived tokens, not long-lived keys



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- **Use Roles:** don't assign permissions to individual users (use groups)
- **Regular Audits:** review who has access to what
- **Service Accounts:** separate identity for applications
- **MFA Everywhere:** especially for privileged accounts
- **Temporary Credentials:** short-lived tokens, not long-lived keys



IAM Best Practices

- **Principle of Least Privilege:** grant only necessary permissions
- **Use Roles:** don't assign permissions to individual users (use groups)
- **Regular Audits:** review who has access to what
- **Service Accounts:** separate identity for applications
- **MFA Everywhere:** especially for privileged accounts
- **Temporary Credentials:** short-lived tokens, not long-lived keys



Outline

- 1 Security Foundations
- 2 Encryption & Data Protection**
- 3 Network & Application Security
- 4 Audit & Compliance



Encryption: *At Rest* vs. *In Transit*

Encryption at Rest

- Data stored on disk
- Protects against theft
- Symmetric keys (AES)
- Key management critical

Encryption in Transit

- Data over network
- Protects eavesdropping
- TLS/SSL protocol
- Certificates validate identity

Rule

Always encrypt sensitive data in both directions. Use HTTPS (TLS 1.2+).



Encryption: *At Rest* vs. *In Transit*

Encryption at Rest

- Data stored on disk
- Protects against theft
- Symmetric keys (AES)
- Key management critical

Encryption in Transit

- Data over network
- Protects eavesdropping
- TLS/SSL protocol
- Certificates validate identity

Rule

Always encrypt sensitive data in both directions. Use HTTPS (TLS 1.2+).



Encryption: *At Rest* vs. *In Transit*

Encryption at Rest

- Data stored on disk
- Protects against theft
- Symmetric keys (AES)
- Key management critical

Encryption in Transit

- Data over network
- Protects eavesdropping
- TLS/SSL protocol
- Certificates validate identity

Rule

Always encrypt sensitive data in both directions. Use HTTPS (TLS 1.2+).



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- Use HSM/KMS: Hardware Security Module or Key Management Service
- Rotate keys: change keys periodically (e.g., yearly)
- Separate keys: different keys for different environments
- Audit access: log who accesses encryption keys
- Key escrow: store backup copies of keys in secure locations



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- **Use HSM/KMS:** Hardware Security Module or Key Management Service
- Rotate keys: change keys periodically (*e.g., yearly*)
- Separate keys: different keys for different environments
- Audit access: log who accesses encryption keys
- Never hardcode: keys in code or config files are catastrophic



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- **Use HSM/KMS:** Hardware Security Module or Key Management Service
- **Rotate keys:** change keys periodically (*e.g., yearly*)
- **Separate keys:** different keys for different environments
- **Audit access:** log who accesses encryption keys
- **Never hardcode:** keys in code or config files are catastrophic



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- **Use HSM/KMS:** Hardware Security Module or Key Management Service
- **Rotate keys:** change keys periodically (*e.g., yearly*)
- **Separate keys:** different keys for different environments
- **Audit access:** log who accesses encryption keys
- **Never hardcode:** keys in code or config files are catastrophic



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- **Use HSM/KMS:** Hardware Security Module or Key Management Service
- **Rotate keys:** change keys periodically (*e.g., yearly*)
- **Separate keys:** different keys for different environments
- **Audit access:** log who accesses encryption keys
- **Never hardcode:** keys in code or config files are catastrophic



Key Management

Critical Problem

If your **encryption key** is stolen, encryption is useless. Key management is **the hardest part** of security.

Best Practices:

- **Use HSM/KMS:** Hardware Security Module or Key Management Service
- **Rotate keys:** change keys periodically (*e.g., yearly*)
- **Separate keys:** different keys for different environments
- **Audit access:** log who accesses encryption keys
- **Never hardcode:** keys in code or config files are catastrophic



Outline

- 1 Security Foundations
- 2 Encryption & Data Protection
- 3 Network & Application Security**
- 4 Audit & Compliance



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- VPC/Subnets: separate public and private networks
- Security Groups: firewall rules per resource
- Network ACL: network-level filtering
- VPN: secure tunnel for private networks
- DMZ: isolated zone for public services
- VPN encrypted tunnel for remote access



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- **VPC/Subnets:** separate public and private networks
- Security Groups: firewall rules per resource
- Network ACLs: subnet-level filtering
- Bastion Hosts: jump server for private access
- VPN: encrypted tunnel for remote access



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- **VPC/Subnets**: separate public and private networks
- **Security Groups**: firewall rules per resource
- Network ACLs: subnet-level filtering
- Bastion Hosts: jump server for private access
- VPN: encrypted tunnel for remote access



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- **VPC/Subnets**: separate public and private networks
- **Security Groups**: firewall rules per resource
- **Network ACLs**: subnet-level filtering
- **Bastion Hosts**: jump server for private access
- **VPN**: encrypted tunnel for remote access



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- **VPC/Subnets**: separate public and private networks
- **Security Groups**: firewall rules per resource
- **Network ACLs**: subnet-level filtering
- **Bastion Hosts**: jump server for private access
- **VPN**: encrypted tunnel for remote access



Network Segmentation

Zero Trust Architecture

Assume network is always compromised. Verify every request, segment traffic, monitor everything.

Techniques:

- **VPC/Subnets**: separate public and private networks
- **Security Groups**: firewall rules per resource
- **Network ACLs**: subnet-level filtering
- **Bastion Hosts**: jump server for private access
- **VPN**: encrypted tunnel for remote access



Application Security

- **OWASP Top 10:** common web **vulnerabilities** (injection, XSS, CSRF)
- **Input Validation:** **sanitize** all user *input*
- **Rate Limiting:** prevent *brute force* and DDoS
- **WAF:** **Web Application Firewall** for HTTP attacks
- **Secrets Management:** never **commit credentials** to git
- **Dependency Scanning:** check for known vulnerabilities in libraries



Outline

- 1 Security Foundations
- 2 Encryption & Data Protection
- 3 Network & Application Security
- 4 Audit & Compliance**



Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- Access logs: who accessed which resources
- API calls: all infrastructure changes
- Authentication: login attempts, MFA usage
- Data access: sensitive data read/write operations
- Configuration changes: who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- API calls: all infrastructure changes
- Authentication: login attempts, *MFA* usage
- Data access: sensitive data read/write operations
- Configuration changes: who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- Authentication: login attempts, *MFA* usage
- Data access: sensitive data read/write operations
- Configuration changes: who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- **Authentication:** login attempts, *MFA* usage
- **Data access:** sensitive data read/write operations
- **Configuration changes:** who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- **Authentication:** login attempts, *MFA* usage
- **Data access:** sensitive data read/write operations
- **Configuration changes:** who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- **Authentication:** login attempts, *MFA* usage
- **Data access:** sensitive data read/write operations
- **Configuration changes:** who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- **Authentication:** login attempts, *MFA* usage
- **Data access:** sensitive data read/write operations
- **Configuration changes:** who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Audit Logging

Accountability

Log everything. Who did what, when, from where. Enables forensics and compliance proof.

What to Log:

- **Access logs:** who accessed which resources
- **API calls:** all infrastructure changes
- **Authentication:** login attempts, *MFA* usage
- **Data access:** sensitive data read/write operations
- **Configuration changes:** who modified what settings

Compliance

Different regulations (GDPR, HIPAA, PCI-DSS) have different audit requirements.

Outline

- 1 Security Foundations
- 2 Encryption & Data Protection
- 3 Network & Application Security
- 4 Audit & Compliance



Recap: Module 7

- **Shared Responsibility:** both provider and customer responsible for security
- **Authentication:** verify identity (who are you?)
- **Authorization:** grant permissions (what can you do?)
- **Encryption:** protect data at rest and in transit
- **Key Management:** the hardest part; use HSM/KMS
- **Network Security:** segmentation, zero trust, firewall rules
- **Audit:** log everything for compliance and forensics



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

